

Visual C# mightyZAP API Manual

Force Control version actuator (model name starts with 12Lf) users can use all the following parameters.

The parameter marked **[F/C version only]** applies only to the Force Control version actuator (model name starts with 12Lf) and does not apply to the Position control version actuator.

Position Control version actuator (model name starts with L12 or D12) user can use all parameters except **[F/C version only]** parameters.

• Initialization – COM Port Connection

`void` OpenMightyZap(`String` portname, `int` BaudRate)

Function : port connection to control servo actuator

Parameter :

- `String` portname : Set COM port number string like "COM6".
- `int` BaudRate : Put communication speed among 9600,19200,57600,115200bps (Factory default 57600)

• Termination

`void` CloseMightyZap()

Function : Close(terminate) port connection which controlled servo actuator

Parameter : (N/A)

• Restart

`void` Restart(`int` bID)

Function : Restart actuator system.

Shutdown due to overload, input voltage error, etc. can be restarted.

Parameter :

- `int` bID : Servo ID (1~253), BroadCAST ID(254), Stand-alone ID (0)
(herein after, explanation about servo ID to be omitted)

• Moving to Goal Position

`void` GoalPosition(`int` bID, `short` position)

Function: Move stroke position to the desired Goal position

Parameter :

- `int` bID :servo ID
- `short` position : Desired position (0~4095)

• Moving Speed Adjustment **[F/C Version Only]**

`void` GoalSpeed(`int` bID, `short` speed)

Function: Moving speed adjustment of servo actuator

Parameter :

- `int` bID : servo ID
- `short` speed : Set moving Speed (0~1023)
(0 : Max speed, 1~1023 Speed adjustment (1:low, 1023:high))

• Goal Current Adjustment **[F/C Version Only]**

```
void GoalCurrent(int bID, short current)
```

Function: Limit max current of servo actuator

Parameter :

- int bID : servo ID
- short current: Set Max current limit (0~1600)

- ※ Speed can be lowered when goal current value is adjusted downward.
- ※ The value represents current[mA] and has +/-15% tolerance.
- ※ High Goal current setting may affect to the lifespan of servo actuator. Please refer to the data sheet for proper use..

• Read Present Position

```
int PresentPosition(int bID)
```

Function : Read present stroke position

Parameter :

- int bID : servo ID
- return : Feedback present position (Return '-1' value in case of Timeout)

• Acceleration Control **[F/C Version Only]**

```
void Acceleration(int bID, byte accel)
```

Function: Control acceleration rate of servo actuator

Parameter :

- int bID : servo ID
- byte accel: Acceleration rate value of actuator (1~255)
The higher the value, the faster the acceleration.

• Deceleration Control **[F/C Version Only]**

```
void Deceleration(int bID, byte decel)
```

Function: Control deceleration rate of servo actuator

Parameter :

- int bID : servo ID
- byte decel: Deceleration rate value of actuator (1~255)
The larger the value, the larger the deceleration rate and the slower the deceleration time.

※If it is set too large, overshoot and minute vibration may occur when reaching the final position.

※ If overshoot occurs while changing the basic settings of the servo motor such as Speed and GoalCurrent, you can reduce the deceleration rate.

• Force On / Off

```
void forceEnable(int bID, bool enable)
```

Function : Activate/Inactivate actuator Force (Motor ON/OFF)

Parameter :

- int bID: servo ID /
- bool enable : 0 - Force Off , 1 - Force On

- ※ Even if it is Forced Off, communication will still work.
- ※ If the next position command is given in the Force Off state, the Force is automatically turned on and the position moves.

What is the Force Off?

The actuator lineup having rated load 35N or more, have a characteristic to keep the position due to the mechanical design characteristics without motor power. Therefore, if the servo actuator in the facility needs to maintain a certain position for considerable time, you can extend the life of the actuator by shutting off the motor power with the Force Off command. In this case, the communication is still maintained, and only the motor is powered off. When the position movement command is given again, the force is automatically turned on and the next command is executed.

• Shutdown On / Off

`void SetShutDownEnable(int bID, byte flag)`

Function : Setting for Shutdown function On/Off for respective error

Parameter :

- int bID :servo ID
- byte flag : Put setting value (See the mightyZAP user manual - Alarm shutdown)

• Read Shutdown Status

`int GetShutDownEnable(int bID)`

Function : Read shutdown On/Off status for error

Parameter :

- int bID :servo ID
- return : Feedback setting status value (Return '-1' value in case of Timeout)

• LED Error Indication

`void SetErrorIndicatorEnable(int bID, byte flag)`

Function : Setting for LED alarm indications On/Off for the respective errors.

Parameter :

- int bID :servo ID
- byte flag : Put setting value
- (See mightyZAP user manual - Alarm LED)

• Read LED Error Indication Status

`byte GetErrorIndicatorEnable(int bID)`

Function : Read LED indications On/Off status for error

Parameter :

- int bID :servo ID
- return : Feedback setting status value (Return '-1' value in case of Timeout)

• Read Error

`byte ReadError(int bID)`

Function : Read current error status

Parameter :

- int bID :servo ID
- return : Feedback error status (Return '-1' value in case of Timeout)

• Direct Write to Address

`void Write_Addr(int bID, byte Addr, byte Size, int Data)`

Function : Writes data directly to the certain address of parameter(memory/parameter) in data map.

Parameter :

- int bID :servo ID
- byte Addr : Parameter address to write data
- byte Size : write address byte size
- int Data :Input Data

(See mightyZAP user manual –Data Map)

• Direct Read from Address

`int Read_Addr(int bID, byte Addr, byte Size)`

Function : Read data directly from the certain address of parameter(memory/parameter) in data map.

Parameter :

- int bID :servo ID
- byte Addr : Parameter address to read
- byte Size : byte Size to read
- return Data : Feedback read data (Return '-1' value in case of Timeout)

(See mightyZAP user manual –Data Map)

• Save data to same address in multiple actuators

`void sync_write_data(byte Addr, byte data, byte size)`

Function : Save data to same address in multiple actuators

Parameter :

- byte Addr : Parameter address to write data, See Factor#1 in the table below
- byte data : Input data matrix, Enter factor values as a matrix after factor #2 in the table below
Length + ID1 + Data#1, +Data#2 +ID2 + Data#1 + data#2
- byte Size : Input Data Size
- EX : Servo ID 1 Goal Position : 1023(0x03FF), Servo ID 2 Goal Position : 2047(0x07FF)

HEADER	ID	Size	Command	Factor #1	Factor #2	
				addr	Length	
0xFFFFF	0xFE	0x0a	0x73	0x86	0x02	
Factor #3	Factor #4	Factor #5	Factor #6	Factor #7	Factor #8	Checksum
1> ID	1> Data #1	1>Data #2	2>ID	2> Data #1	2> Data #2	
0x01	0xff	0x03	0x02	0xFF	0x07	0xF1

(For more details, See the mightyZAP user manual –Symmetric Store)

• 1 byte transimssion

`void writeByte(byte Data)`

Function : Send a single data to the actuator. Used when the user configures and sends the packet.

Parameter :

- byte Data : 1byte Data to be transmitted to actuator

• Direct packet transmission

`void WritePacket(byte buffer, byte size)`

Function : Use when the user directly constructs a packet and sends one completed packet to the servo actuator

Parameter :

- **byte** buffer : Packet buffer to be sent to the actuator
 - **byte** size: Packet buffer size to be sent to the actuator
- (See the mightyZAP user manual – Packet Description and Packet examples)